

Instructions

Neeche 5 practice exercises hain — har ek concept ka ek exercise.

Code padhein, output predict karein, aur browser console ya Node.js REPL mein try karein.

Answers neeche diye hain — pehle khud try karein!

Exercise 1 — var vs let vs const Scope Test

Concept: [let/const/var](#) **Instructions:** Neeche diya hua code dekho. Predict karo ki kaunsi line error degi aur kaunsi nahi.

```
function testScope() {  
  if (true) {  
    var x = 'var';  
    let y = 'let';  
    const z = 'const';  
  }  
  console.log(x); // ?  
  console.log(y); // ?  
  console.log(z); // ?  
}  
testScope();
```

Answer / Key Insight

console.log(x) → 'var' (function scope mein hai)

console.log(y) → ReferenceError (block ke bahar access)

console.log(z) → ReferenceError (block ke bahar access)

Exercise 2 — ES5 to Arrow Function Refactor

Concept: [Arrow Functions](#) **Instructions:** Neeche ES5 functions hain. Inhe ES6 arrow functions mein convert karo:

```
// ES5 — Old way
var multiply = function(a, b) { return a * b; };
var isEven = function(n) { return n % 2 === 0; };
var square = function(n) { return { value: n * n }; };

// ■ Your answer (ES6 Arrow):
const multiply = (a, b) => a * b;
const isEven = n => n % 2 === 0;
const square = n => ({ value: n * n });

// Test
console.log(multiply(3, 4)); // 12
console.log(isEven(7)); // false
console.log(square(5)); // { value: 25 }
```

Answer / Key Insight

Single-expression arrow functions mein return keyword ki zaroorat nahi.

Object return karte waqt () zaroori hain — warna {} ko function body samjha jaata hai.

Exercise 3 — Nested Destructuring

Concept: [Destructuring](#) **Instructions:** Ek API response se ek hi line mein specific values extract karo:

```
const apiResponse = {
  user: {
    id: 101,
    name: 'Arjun Sharma',
    email: 'arjun@example.com',
    scores: [85, 92, 78]
  },
  status: 'success'
};

// Extract: userName, score1 (1st), score3 (3rd), status
const {
  user: { name: userName, scores: [score1, , score3] },
  status
} = apiResponse;

console.log(userName); // 'Arjun Sharma'
console.log(score1); // 85
console.log(score3); // 78
console.log(status); // 'success'
```

Answer / Key Insight

Nested destructuring mein colon ke baad dusra pattern likhte hain.

Array mein comma se elements skip karte hain.

Exercise 4 — Spread & Rest Operations

Concept: Spread / Rest **Instructions:** Teen spread/rest tasks complete karo:

```
// Task 1: Do arrays merge karo
const batch1 = ['Alice', 'Bob', 'Charlie'];
const batch2 = ['Dave', 'Eve'];
const allStudents = [...batch1, ...batch2];
// ['Alice', 'Bob', 'Charlie', 'Dave', 'Eve']

// Task 2: Unlimited args lena wala average function
const average = (...marks) =>
marks.reduce((sum, m) => sum + m, 0) / marks.length;
console.log(average(80, 90, 70, 85)); // 81.25

// Task 3: Object clone + update (immutable pattern)
const original = { name: 'Rahul', city: 'Delhi', score: 88 };
const updated = { ...original, score: 95 };
console.log(original.score); // 88 — unchanged!
console.log(updated.score); // 95
```

Answer / Key Insight

Spread se original array/object change nahi hota — ek naya copy banta hai.

Rest operator hamesha last position mein hona chahiye.

Exercise 5 — Template Literal Report Generator

Concept: [Template Literals](#) **Instructions:** Is function ko complete karo jo template literals use karke report card generate kare:

```
const generateReport = ({ name, roll, marks }) => {
  const total = marks.reduce((s, m) => s + m, 0);
  const avg = (total / marks.length).toFixed(1);
  const grade = avg >= 90 ? 'A+' : avg >= 75 ? 'A' : 'B';

  return `
=====
STUDENT REPORT CARD
=====
Name : ${name}
Roll : ${roll}
Marks : ${marks.join(', ')}
Total : ${total} / ${marks.length * 100}
Avg : ${avg}%
Grade : ${grade}
=====`;
};

console.log(generateReport({
  name: 'Priya Verma', roll: 'CS-042',
  marks: [88, 92, 76, 95, 84]
}));
```

Answer / Key Insight

Output: Priya Verma | CS-042 | Total: 435/500 | Avg: 87.0% | Grade: A

Template literals mein expressions directly likh sakte hain — `.toFixed()`, ternary sab.